GRID TRADING MASTERY · PART 9

Python Implementation Guide

Project Structure · Core Modules · Bybit API Integration

Backtesting Framework · Paper Trading → Live Migration

การ deploy Bot บน Server + Monitoring

แก่นของ PART นี้

Part นี้นำ pseudocode จาก Part 1–8 มาประกอบเป็น Python project ที่ runnable ได้จริง ทุกโค้ดที่เขียนในเล่มนี้ออกแบบมาให้ต่อกันได้ใน architecture ที่แสดงด้านล่าง

9.1 Project Structure

```
grid-trading-bot/ ├── core/ │ ├── __init__.py │ ├── grid_framework.py
# Zone/Step/Capital/Risk (Part 1A, 3, 5) │ ├── state_machine.py #
Regime Detection (Part 4) │ ├── execution_styles.py # 4 Execution
Styles (Part 7B) │ └── dd_monitor.py # Drawdown Monitor (Part 5) ├──
strategies/ │ ├── buy_only.py # Part 1A │ ├── bidirectional.py # Part
1B │ ├── hedge_grid.py # Part 1C │ ├── pair_grid.py # Part 6 │ └──
snowball.py # Part 6B ├── indicators/ │ ├── hurst.py # Hurst Exponent
(Part 4) │ ├── atr_donchian.py # ATR, Donchian, Keltner (Part 3) │ ├──
composite_score.py # RSI/BB/Vol/Hurst score (Part 3B) │ └── kelly.py #
Kelly Criterion (Part 5) ├── exchange/ │ ├── bybit_client.py # Bybit
API wrapper │ └── data_feed.py # Market data fetcher ├── backtest/ │
├── backtest_engine.py # Event-driven backtester │ ├── metrics.py #
Sharpe, Calmar, Max DD │ └── bootstrap_zone.py # Block Bootstrap (Part
3) ├── monitoring/ │ ├── watchdog.py # Grid Watchdog (Part 4) │ ├──
alerts.py # LINE/Telegram notify │ └── dashboard.py # Simple web
dashboard ├── config/ │ ├── config.yaml # All parameters │ └──
secrets.yaml # API keys (gitignored) ├── tests/ │ └── test_*.py # Unit
tests ├── main.py # Entry point └── requirements.txt
```

9.2 Core Grid Framework

```
# core/grid_framework.py from dataclasses import dataclass, field from
typing import List, Optional import numpy as np @dataclass class
GridLevel: price: float qty: float side: str # "BUY" or "SELL"
tp_price: float sl_price: Optional[float] = None order_id:
Optional[str] = None filled: bool = False @dataclass class
GridFramework: """ Core grid parameters — computed once per deployment
""" symbol: str zone_high: float zone_low: float step: float
grid_capital: float base_q: float # Q per level (Kelly-based, Part 5)
n_levels: int = field(init=False) avg_price: float = field(init=False)
def __post_init__(self): self.n_levels = int((self.zone_high -
self.zone_low) / self.step) self.avg_price = (self.zone_high +
self.zone_low) / 2 @classmethod def from_market_data(cls, symbol,
prices, highs, lows, grid_capital, kelly_fraction=0.19): """ สร้าง
GridFramework จากข้อมูลตลาด (Part 3 + Part 5) """ from
indicators.atr_donchian import compute_atr, compute_donchian
current_price = prices[-1] # Zone from Donchian (Part 3) dc_high,
dc_low = compute_donchian(highs, lows, period=30) zone_high = dc_high
* 1.05 zone_low = dc_low * 0.95 # Step from ATR (Part 3) atr =
compute_atr(highs, lows, prices, period=14) step = max(round(0.5 * atr
/ 100) * 100, 3 * current_price * 0.001 * 2) # Q from Kelly-sized
capital (kelly_fraction already applied to net_worth before call) #
grid_capital = net_worth × kelly_fraction → Q reflects Kelly sizing
implicitly n = int((zone_high - zone_low) / step) avg_p = (zone_high +
zone_low) / 2 base_q = grid_capital / (n * avg_p) return
cls(symbol=symbol, zone_high=zone_high, zone_low=zone_low, step=step,
grid_capital=grid_capital, base_q=round(base_q, 4)) def
create_buy_levels(self, current_price, sizes=None) -> List[GridLevel]:
"""วาง BUY orders ต่ำกว่า current_price""" levels = [] price =
current_price - self.step i = 0 while price >= self.zone_low: q =
sizes[i] if sizes else self.base_q levels.append(GridLevel(
price=round(price, 0), qty=q, side="BUY", tp_price=round(price +
self.step, 0) )) price -= self.step i += 1 return levels def
allocate_by_weight(self, style, size_mult=1.0) -> List[float]:
"""คำนวณ sizes ตาม allocation style (Part 2)""" import math n =
self.n_levels if style == "equal": weights = [1.0] * n elif style ==
"bell": center = n // 2 weights = [math.exp(-0.5 * ((i-
center)/(n/4))**2) for i in range(n)] elif style == "bottom_heavy":
weights = [1.0/(i+1) for i in range(n)] else: weights = [1.0] * n
```

```
total = sum(weights) return [self.base_q * (w/total) * n * size_mult
for w in weights]
```

9.3 Bybit API Integration

```
# exchange/bybit_client.py import hmac import hashlib import time,
requests from typing import Optional class BybitClient: BASE_URL =
"https://api.bybit.com" def __init__(self, api_key: str, api_secret:
str, testnet: bool = True): self.api_key = api_key self.api_secret =
api_secret if testnet: self.BASE_URL = "https://api-testnet.bybit.com"
# ⚠️ Security: Never log api_secret or the resulting signature. # Load
api_secret from env var only: os.environ["BYBIT_API_SECRET"] def
_sign(self, params: dict, timestamp: int) -> str: query = "&".join(f"
{k}={v}" for k, v in sorted(params.items())) msg = f"{timestamp}
{self.api_key}5000{query}" return hmac.new(self.api_secret.encode(),
msg.encode(), hashlib.sha256).hexdigest() def _request(self, method:
str, endpoint: str, params: dict = None): ts = int(time.time() * 1000)
params = params or {} sig = self._sign(params, ts) headers = { "X-
BAPI-API-KEY": self.api_key, "X-BAPI-SIGN": sig, "X-BAPI-TIMESTAMP":
str(ts), "X-BAPI-RECV-WINDOW": "5000", } url = self.BASE_URL +
endpoint if method == "GET": resp = requests.get(url, params=params,
headers=headers) else: resp = requests.post(url, json=params,
headers=headers) return resp.json() def place_order(self, symbol: str,
side: str, qty: float, price: float, order_type: str = "Limit") ->
dict: params = { "category": "spot", "symbol": symbol, "side": side,
"orderType": order_type, "qty": str(qty), "price": str(price),
"timeInForce": "GTC", # Good Till Cancel } return
self._request("POST", "/v5/order/create", params) def
cancel_all_orders(self, symbol: str) -> dict: return
self._request("POST", "/v5/order/cancel-all", {"category": "spot",
"symbol": symbol}) def get_orderbook(self, symbol: str, limit: int =
5) -> dict: return self._request("GET", "/v5/market/orderbook",
{"category": "spot", "symbol": symbol, "limit": limit}) def
get_wallet_balance(self, coin: str = "USDT") -> float: resp =
self._request("GET", "/v5/account/wallet-balance", {"accountType":
"UNIFIED"}) for item in resp.get("result", {}).get("list", [{}])
[0].get("coin", []): if item["coin"] == coin: return
float(item["walletBalance"]) return 0.0
```

9.4 Backtesting Engine

```
# backtest/backtest_engine.py import pandas as pd import numpy as np
from core.grid_framework import GridFramework, GridLevel class
GridBacktester: """ Event-driven backtester สำหรับ Buy-Only Spot Grid
""" def __init__(self, grid: GridFramework, fee_rate: float = 0.001):
self.grid = grid self.fee_rate = fee_rate def run(self, ohlcv_df:
pd.DataFrame) -> dict: """ ohlcv_df: DataFrame ที่มีคอลัมน์ open, high,
low, close, volume """ capital = self.grid.grid_capital usdt = capital
# เริ่มด้วย USDT ทั้งหมด btc = 0.0 # ไม่มี BTC เริ่มต้น inventory = {} #
{buy_price: qty} trades = [] portfolio_values = [] # สร้าง BUY levels
current_price = ohlcv_df["close"].iloc[0] buy_levels =
self.grid.create_buy_levels(current_price) for idx, row in
ohlcv_df.iterrows(): low_price = row["low"] high_price = row["high"]
close = row["close"] # หมายเหตุ: logic นี้เป็น optimistic assumption —
ราคา candle แตะ level ไม่ได้หมายความว่า fill ทุกครั้ง # ใน production ให้ใช้
WebSocket order events แทน # Check BUY fills (price ลงถึง BUY level)
for level in buy_levels: if not level.filled and low_price <=
level.price: cost = level.qty * level.price * (1 + self.fee_rate) if
usdt >= cost: usdt -= cost btc += level.qty inventory[level.price] =
level.qty level.filled = True trades.append({"date": idx, "side":
"BUY", "price": level.price, "qty": level.qty}) # Check SELL fills (TP
hit) for buy_price, qty in list(inventory.items()): tp_price =
buy_price + self.grid.step if high_price >= tp_price: proceeds = qty *
tp_price * (1 - self.fee_rate) usdt += proceeds btc -= qty del
inventory[buy_price] # Reset BUY level for level in buy_levels: if
level.price == buy_price: level.filled = False trades.append({"date":
idx, "side": "SELL", "price": tp_price, "qty": qty}) # Portfolio value
portfolio_values.append(usdt + btc * close) # Metrics pv =
pd.Series(portfolio_values, index=ohlcv_df.index) returns =
pv.pct_change().dropna() max_dd = (pv / pv.cummax() - 1).min() sharpe
= returns.mean() / returns.std() * np.sqrt(365) if returns.std() > 0
else 0 return { "final_portfolio": portfolio_values[-1],
"total_return": (portfolio_values[-1] / capital - 1), "max_drawdown":
max_dd, "sharpe_ratio": sharpe, "n_trades": len(trades), "trades":
pd.DataFrame(trades), "portfolio_values": pv, }
```

9.5 Main Entry Point + Config

```
# config/config.yaml grid: symbol: "BTCUSDT" grid_capital: 20000
kelly_fraction: 0.19 # f_safe = 25% × f* ≈ 19% (ตาม Part 5 ตัวอย่าง
f*=0.755) donchian_period: 30 atr_period: 14 step_factor: 0.5 # Step =
step_factor × ATR risk: dd_halt_threshold: -0.08 # −8% drawdown → HALT
(Part 5) dd_resume_threshold: -0.04 # −4% → resume
max_notional_per_level: 0.05 # 5% of capital (Part 5) regime:
hurst_on: 0.50 # Grid ON ถ้า H < 0.50 hurst_caution: 0.58 # CAUTION ถ้า
H 0.50–0.58 adx_caution: 25 adx_off: 35 bb_width_caution: 0.04
bb_width_off: 0.08 execution: style: "composite" # mean_reversion /
trend_adaptive / volume_adaptive / composite recheck_hours: 1 # ตรวจ
regime ทุก 1 ชั่วโมง snowball: enabled: true type: 1 # 1=simple compound,
2=triggered layer, 3=zone upgrade dca_pct: 0.30 # 30% ของ profit ไป
DCA # === ไฟล์แยก: main.py === import yaml from core.grid_framework
import GridFramework from core.state_machine import GridStateMachine
from core.dd_monitor import DrawdownMonitor from core.execution_styles
import UnifiedGridController from exchange.bybit_client import
BybitClient from exchange.data_feed import DataFeed from
monitoring.watchdog import GridWatchdog import time, logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%
(levelname)s] %(message)s") def main(): # Load config with
open("config/config.yaml") as f: cfg = yaml.safe_load(f) with
open("config/secrets.yaml") as f: secrets = yaml.safe_load(f) #
Alternative (recommended): load from env vars instead of secrets.yaml
# api_key = os.environ["BYBIT_API_KEY"] # api_secret =
os.environ["BYBIT_API_SECRET"] # Initialize client =
BybitClient(secrets["api_key"], secrets["api_secret"], testnet=True)
data = DataFeed(client, cfg["grid"]["symbol"]) # Get market data
prices, highs, lows, volumes = data.get_ohlcv(days=60) current_price =
prices[-1] # Build framework framework =
GridFramework.from_market_data( symbol=cfg["grid"]["symbol"],
prices=prices, highs=highs, lows=lows, grid_capital=cfg["grid"]
["grid_capital"], kelly_fraction=cfg["grid"].get("kelly_fraction",
0.19) ) logging.info(f"Grid: zone=[{framework.zone_low:.0f},
{framework.zone_high:.0f}] " f"step={framework.step:.0f} Q=
{framework.base_q:.4f} N={framework.n_levels}") # Controller
controller = UnifiedGridController(framework, style=cfg["execution"]
["style"]) watchdog = GridWatchdog(controller,
check_interval_hours=cfg["execution"]["recheck_hours"]) # Main loop
```

```
portfolio_history = [cfg["grid"]["grid_capital"]] while True:
market_data = data.get_indicators(prices, highs, lows, volumes)
market_data["portfolio_history"] = portfolio_history result =
watchdog.run_checks(market_data) logging.info(f"State:
{result['state']} | Actions: {result['actions']}") for action in
result["actions"]: if action["action"] == "CANCEL_ALL_ORDERS":
client.cancel_all_orders(cfg["grid"]["symbol"])
logging.warning(f"Cancelled all orders: {action['reason']}") elif
action["action"] == "PLACE_ORDERS": for order in action.get("orders",
[]): resp = client.place_order( symbol=cfg["grid"]["symbol"],
side=order["side"], qty=order["qty"], price=order["price"] )
logging.info(f"Order placed: {resp}") # Update portfolio value balance
= client.get_wallet_balance("USDT") portfolio_history.append(balance)
# ⚠️ Production Note: sleep(3600) จะพลาด order fills ระหว่างรอ # ใช้
WebSocket private channel (wss://stream.bybit.com/v5/private) แทน
polling # ดู Bybit API docs: subscribe to "order" channel เพื่อรับ fill
events real-time # ⚠️ Production: implement exponential back-off
reconnect — connection drop = missed fills # while True: try:
ws.run_forever() except: time.sleep(min(2**attempt, 60)); attempt += 1
time.sleep(cfg["execution"]["recheck_hours"] * 3600) if __name__ ==
"__main__": main()
```

9.6 Paper Trading → Live Migration

Phase	ระยะเวลา	ทำอะไร	Success Criteria
Phase 0: Backtest	1 สัปดาห์	Run backtester บน 1–2 ปีย้อนหลัง	Sharpe > 1.5, Max DD < 15%
Phase 1: Paper Trade	2–4 สัปดาห์	Run bot บน Testnet (สภาพแวดล้อมทดสอบของ Bybit — API จริง แต่เงินจำลอง ตั้ง testnet=True)	ไม่มี bug, state machine ทำงาน
Phase 2: Small Live	1 เดือน	$1,000 จริง บน Bybit	P&L ≈ backtest, DD ≤ −8%
Phase 3: Scale Up	ต่อเนื่อง	เพิ่ม capital หลังผ่าน Phase 2	Compound + DCA Stack

9.7 Pre-Live Checklist — สิ่งที่ต้องตรวจก่อน Live

ตรวจทุกข้อก่อน switch testnet=False

- API Rate Limit: Bybit อนุญาต 600 requests/นาที → grid 30 levels × 2 (place/cancel) = 60/cycle ✓
- Minimum Order Size: BTC บน Bybit ขั้นต่ำ 0.001 BTC → Q ต้องมากกว่านี้
- Price Precision: BTC round ถึง $0.01 → step ต้องเป็น multiple ของ $0.01
- Network Redundancy: รัน bot บน VPS (Virtual Private Server — เซิร์ฟเวอร์เช่าที่ online ตลอด 24/7) พร้อม reconnect logic ไม่ควรรัน bot บน local machine ที่อาจปิดหรือ internet หลุด
- Secret Management: API key ใน environment variable ไม่ใช่ hardcode

9.3 Monitoring Dashboard

```
# monitoring/dashboard.py (Flask simple dashboard) from flask import
Flask, jsonify, render_template_string import json app =
Flask(__name__) HTML = """
```

Grid Bot Dashboard

```
Loading...

""" @app.route("/") def dashboard(): return
render_template_string(HTML) @app.route("/api/status") def status(): #
อ่านจาก state file ที่ bot เขียนไว้ try: with open("state.json") as f:
return jsonify(json.load(f)) except FileNotFoundError: return
jsonify({"error": "Bot not running"}) # รัน: python -m
monitoring.dashboard if __name__ == "__main__":
app.run(host="0.0.0.0", port=8080)
```

สรุปเนื้อหาทั้งเล่ม — Cheat Sheet

```
GRID TRADING MASTERY — Quick Reference ZONE: Donchian(30) ± 5%
buffer | min_width ≥ 6×ATR STEP: max(0.5×ATR(14),
3×fee_per_cycle) → round to $100 Q: grid_capital / (N ×
avg_price) → ตรวจ 5% max notional N: zone_width / step → ควรอยู่
ระหว่าง 15–25 KELLY: f* = (p×b − q)/b | f_safe = 0.25 × f*
grid_capital = net_worth × f_safe REGIME (Part 4, BTC-
calibrated): H < 0.50 + ADX < 40 + BB_width < 4% → GRID ON (full
deploy) H 0.50–0.58 + ADX 40–50 + ... → CAUTION (50% size) H >
0.58 + ADX > 50 + BB_width > 8% → GRID OFF (pause) RISK (Part
5): DD ≥ −8% → HALT (cancel all, wait for recovery) DD ≥ −4% + H
< 0.55 + 24h cooloff → RESUME SNOWBALL (Part 6B): Type 1: Q(t) =
Q₀ × (1 + r_daily)^t DCA Stack: 30% profit → weekly BTC buy
COMPOSITE SCORE (Part 3B): 30%×RSI + 30%×BB + 20%×Vol +
```

```
20%×Hurst → 0–100 >75: Full Deploy | 55–75: 60% | 35–55: 30% |
<35: Wait
```

แบบฝึกหัด Part 9

▶ ข้อ 1: อ่าน config.yaml แล้วอธิบายว่า kelly_fraction=0.19 หมายความว่าอย่างไร และ grid_capital=$20,000 มาจากการคำนวณอย่างไร?

▶ ข้อ 2: ใน run_cycle() ถ้า DD = −9% เกิดอะไรขึ้น? bot ยังคงทำงานไหม?

▶ ข้อ 3: ทำไม testnet=True ต้องถูกเปลี่ยนเป็น False ก่อน Live? มีความเสี่ยงอะไรถ้าลืม?

▶ ข้อ 4: Backtester ใน Part 9 มี "optimistic assumption" อะไรที่ทำให้ผลดีกว่าความเป็นจริง?